

LunarSight: A Multi-Agent Software Architecture for Lunar Subsurface Ice Detection and Autonomous Traverse Planning

The transition from orbital remote sensing to sustained in-situ resource utilization (ISRU) on the lunar surface requires a paradigm shift in how planetary data is processed, synthesized, and operationalized. Historically, lunar radar astronomy relied on Earth-based observations from facilities like the Arecibo Observatory or single-frequency orbital instruments such as the Mini-RF on the Lunar Reconnaissance Orbiter (LRO) and Chandrayaan-1. These legacy systems predominantly operated in the S-band (12 cm wavelength) utilizing a hybrid-polarimetric architecture, which, while effective for surficial characterization, severely limited subsurface penetration.

The introduction of the Chandrayaan-2 Dual Frequency Synthetic Aperture Radar (DFSAR) instrument represents a quantum leap in microwave imaging. By operating simultaneously in both the L-band (24 cm wavelength) and the S-band, DFSAR offers the first fully polarimetric (quad-pol) observations of the Moon from orbit. The longer L-band wavelength penetrates several meters into the cryogenic regolith, allowing for the differentiation between thin surficial frost layers and massive, deeply buried volumetric ice slabs. However, isolating these volatile signatures within the doubly shadowed terrain of the South Polar Regions—such as the extreme environment of the Faustini Crater—demands multi-modal data synthesis, complex radar polarimetry, and kinodynamically aware pathfinding.

To address these monumental computational challenges, the "LunarSight" architecture operates as a sophisticated, five-agent artificial intelligence ecosystem. By synthesizing DFSAR data with high-resolution topographic models, the system autonomously isolates volumetric ice signatures and routes rovers through extreme cryogenic terrain. This report provides an exhaustive, seven-chapter technical deconstruction of the LunarSight system, detailing the theoretical foundations, technological stacks, internal operational flows, orchestration methodologies, and strategic blueprints required to scale this architecture into a mission-ready deployment.

Chapter 1: Agent 1 - The Data Ingestion and Topographic Synthesis Agent

The foundational layer of the LunarSight architecture relies on the precise acquisition, spatial alignment, and mathematical synthesis of orbital data. Agent 1 is explicitly designed to interface with planetary data archives, extracting raw microwave and altimetry arrays, and generating the baseline geospatial tensors required by all downstream analytical agents. Without a geometrically flawless ingestion pipeline, subsequent deep learning and polarimetric calculations will suffer from severe spatial misalignment, rendering autonomous navigation impossible.

Pipeline and Technological Stack

Agent 1 functions as an automated spatial data engineering pipeline. It requires programmatic, API-driven access to the ISRO Science Data Archive (ISDA) via the PRADAN (Policy Based Data Retrieval, Analytics, Dissemination and Notification) portal. This portal serves as the primary repository for Chandrayaan-2 data, housing the PDS4-compliant DFSAR imagery. The agent specifically targets the L-band and S-band complex radar swathes, prioritizing data collected over the permanently shadowed regions of the lunar South Pole.

Simultaneously, the agent interfaces with the NASA Planetary Data System (PDS) LOLA Data Node. The Lunar Orbiter Laser Altimeter (LOLA) provides global topographic mapping with a vertical precision of approximately 10 centimeters and an accuracy of 1 meter. For high-latitude operations poleward of 60° , LOLA produces the most accurate polar terrain models available, providing the necessary geodetic framework to control the radar data.

The primary technological stack driving Agent 1 includes the Geospatial Data Abstraction Library (GDAL) and its Python binding, Rasterio. These are industry-standard C/C++ libraries capable of warping, projecting, and reading massive multi-gigabyte spatial raster arrays in memory-efficient blocks. NumPy and SciPy are heavily utilized for vectorized matrix transformations and array manipulation, while custom PDS4 client libraries are deployed to parse the complex XML labels inherent to the international standard for planetary science data archiving.

Topographic Synthesis and Horn's Algorithm

Raw elevation data, while valuable, is insufficient for direct rover navigation; it must be converted into deterministic slope gradients. Agent 1 is programmed to compute localized slope using Horn's algorithm, a mathematically robust estimator that evaluates a 3×3 moving window across the Digital Elevation Model.

The algorithm calculates the east-west gradient ($\frac{\partial z}{\partial x}$) and the north-south gradient ($\frac{\partial z}{\partial y}$) using finite difference approximations. The east-west gradient describes how elevation changes between the east and west sides of the pixel neighborhood, weighted heavily toward the central row :

$$\partial z / \partial x = ((z_{i+1, j-1} + 2z_{i+1, j} + z_{i+1, j+1}) - (z_{i-1, j-1} + 2z_{i-1, j} + z_{i-1, j+1})) / 8\Delta x$$

Similarly, the north-south gradient describes elevation changes across the vertical axis :

$$\partial z / \partial y = ((z_{i-1, j+1} + 2z_{i, j+1} + z_{i+1, j+1}) - (z_{i-1, j-1} + 2z_{i, j-1} + z_{i+1, j-1})) / 8\Delta y$$

The overall surface slope angle (θ) is then derived by combining these gradients :

$$\text{Slope } (\theta) = \arctan(\sqrt{(\partial z / \partial x)^2 + (\partial z / \partial y)^2})$$

Furthermore, the aspect (the compass direction that the slope faces) is calculated using the arctangent of the two gradients, providing critical azimuth data for the rover's solar panel orientation. By integrating Horn's method natively into the data pipeline, Agent 1 ensures that downstream terramechanics agents receive highly accurate, mathematically smoothed gradient tensors. This method is vastly superior to simpler algorithms (such as Zevenbergen-Thorne or unweighted nearest-neighbor models) for highly cratered lunar terrains, as the weighted 3×3 window effectively suppresses high-frequency noise and artificial stepping anomalies inherent in laser altimetry.

Internal Operational Flow

The internal execution architecture of Agent 1 follows a strict, step-by-step sequential flow designed for autonomous fault tolerance.

The initiation phase is triggered by a geospatial bounding box passed via the system's global state object, defining the target latitude and longitude parameters (e.g., the coordinates for the Faustini Crater at roughly 87.3° S, 77° E). Upon receiving these coordinates, the agent authenticates with the ISDA PRADAN and NASA PDS APIs, querying the databases for available radar and elevation datasets intersecting the bounding box.

During the execution phase, the agent downloads the relevant PDS4 .img and .tif data products into localized, high-speed NVMe storage to prevent I/O bottlenecks. Utilizing GDAL, the agent reprojects all spatial arrays into a unified lunar polar stereographic coordinate reference system, referenced to a 1737.4 kilometer lunar sphere, ensuring that a pixel in the radar array perfectly aligns with the same physical square meter in the topographic array. The LOLA DEM is then processed through the vectorized Horn's algorithm to generate the discrete slope and aspect arrays.

In the final output phase, the agent writes a highly co-registered, multi-channel spatial tensor to the system's shared memory. This tensor contains the raw S-band complex data, L-band complex data, LOLA elevation data, and Horn-derived slope gradients. The agent concludes its operation by updating the global state with a status flag indicating ingestion success, thereby signaling the orchestration layer to proceed.

Learning Resource Category	Specific Reference / Link	Application within Agent 1 Pipeline
Planetary Data Standards	Chandrayaan-2 IIRS PDS4 Data Products User Guide	Parsing XML metadata and unpacking ISDA .img complex radar files.
Topographic Mathematics	Modeling Slope in a Geographic Information System (Weih & Mattson)	Theoretical justification for deploying Horn's algorithm over alternative spatial derivatives.
Geospatial Processing	GDAL/Rasterio Official Documentation	Managing affine transformations, reprojections, and in-memory raster array manipulation.

Chapter 2: Agent 2 - The Complex-Valued SAR Despeckling Agent

Synthetic Aperture Radar imaging is inherently corrupted by speckle. This spatial degradation is not additive noise, but rather a spatially correlated, signal-dependent multiplicative noise originating from the constructive and destructive interference of coherent backscatter generated by sub-resolution scatterers within each pixel footprint. Agent 2 is a dedicated, highly specialized computer vision model engineered to mathematically strip this interference from the DFSAR data without compromising the foundational physics required for downstream polarimetric decomposition.

Pipeline and Technological Stack

The standard multiplicative model for SAR speckle is expressed as $Y = F \cdot X$, where Y is the observed noisy intensity, X is the underlying noise-free radar cross-section, and F represents

the normalized fading speckle random variable. Historically, the mitigation of speckle relied on multilook integration and spatial filtering techniques, such as the Lee, Frost, Refined Lee, and Gamma-MAP filters. These classical filters operate by calculating local statistics within a moving window. While computationally trivial, they introduce severe systemic flaws into high-resolution planetary workflows: they aggressively blur sharp topographic edges, smooth over fine structural textures, and, most critically, frequently destroy the relative polarimetric phase information required for calculating Stokes parameters.

To circumvent these limitations, Agent 2 pivots toward the vanguard of modern radar processing by utilizing a Complex-Valued Convolutional Neural Network (CV-CNN) designed as a denoising autoencoder. Standard Real-Valued (RV) neural networks process only the amplitude (intensity) of the radar returns, permanently discarding the complex phase. In stark contrast, CV-CNNs utilize specialized complex convolutional layers, complex activation functions (such as Complex-ReLU or Complex-Sigmoid), and complex batch normalization protocols. This architecture inherently preserves the complete phase matrix of the electromagnetic wave, a requirement for fully polarimetric analysis.

The technological backbone for Agent 2 is PyTorch, specifically augmented by libraries such as ComplexPyTorch and cvnn. Since PyTorch version 1.7, the framework natively supports the `torch.complex64` datatype, allowing tensors to maintain distinct real and imaginary components throughout the forward pass and backpropagation processes, facilitating the calculation of complex gradients. The NVIDIA CUDA ecosystem is heavily relied upon to accelerate these computationally intense floating-point operations.

Complex Covariance Matrix Preservation

The Chandrayaan-2 DFSAR instrument operates in both fully polarimetric (quad-pol) and compact polarimetric modes, transmitting and receiving varying combinations of electromagnetic polarizations (e.g., Horizontal-Horizontal, Horizontal-Vertical). The backscattered signal measured by the antenna is represented mathematically by a complex scattering matrix. Because the lunar surface consists of distributed scatterers (regolith, rocks, ice), this scattering matrix is statistically transformed into a positive semi-definite complex covariance matrix (C_2 for dual-pol, or C_3 for quad-pol).

For a dual-polarization system, the 2×2 covariance matrix C_2 is defined by the following expectation values:

$$C_2 = \begin{bmatrix} \langle |E_{\{HH\}}|^2 \rangle & \langle E_{\{HH\}} E_{\{HV\}}^* \rangle \\ \langle E_{\{HV\}} E_{\{HH\}}^* \rangle & \langle |E_{\{HV\}}|^2 \rangle \end{bmatrix}$$

The diagonal elements of the C_2 matrix represent the real-valued intensity of the radar backscatter for each specific polarization. Crucially, the off-diagonal elements encode the complex cross-covariance terms, capturing the critical phase correlation properties ($\phi_{\{HH-HV\}}$) between the channels.

Agent 2's CV-CNN autoencoder is engineered to map the noisy complex covariance matrix to a clean, idealized target matrix. By minimizing a complex-aware loss function, the network isolates the multiplicative speckle distribution and strips it away. Because the convolutions operate in the complex domain, the integrity of the complex conjugates in the off-diagonal elements is perfectly maintained.

Internal Operational Flow

The operational sequence of Agent 2 is computationally intensive, transforming raw geospatial data into pristine physical matrices.

The initiation phase begins when the agent reads the state object to locate the co-registered raw L-band and S-band complex arrays generated by Agent 1. Upon confirming the integrity of the files, the agent initializes the CV-CNN architecture in PyTorch, loading pre-trained weights optimized for radar cross-section extraction, and moves the model parameters onto the available cloud GPU architecture (e.g., an NVIDIA H100 or A100). Execution commences by mathematically transforming the raw multi-look complex data into the corresponding C_2 and C_3 covariance matrices for both the L-band and S-band frequencies. These matrices are converted into torch.complex64 tensors and fed into the CV-CNN. The network passes the tensors through successive complex 2D and 3D convolutions, extracting high-dimensional latent features that represent the underlying lunar topology, while systematically filtering out the multiplicative noise distribution in the complex domain. The output phase produces a mathematically pristine, noise-free complex covariance tensor. The agent verifies that the polarimetric phase relationships between the transmit and receive channels remain intact, serializes the clean tensor to disk, and updates the global graph state to trigger the feature extraction processes.

Learning Resource Category	Specific Reference / Link	Application within Agent 2 Pipeline
Deep Learning for Radar	Complex-Valued Vs. Real-Valued Convolutional Neural Network for PolSAR Data Classification	Theoretical justification for preserving phase data via CV-CNNs over standard RV-CNNs.
PyTorch Complex Tensors	ComplexPyTorch GitHub Repository	Implementation of complex layers, activations, and complex backpropagation mechanisms.
SAR Physics	SAR Image Despeckling Based on Convolutional Denoising Autoencoder	Understanding the multiplicative noise model and latent feature extraction in autoencoders.

Chapter 3: Agent 3 - The Polarimetric Feature Extraction Agent

With the speckle interference surgically removed, the data exists as a pristine, complex mathematical representation of the radar echo. However, this raw covariance matrix is not directly actionable by computer vision segmentation models. Agent 3 serves as a deterministic physics engine, translating the complex electrical field measurements into distinct physical geological indicators, calculating the specific polarimetric decompositions required to differentiate between jagged surface roughness and volumetric subsurface water-ice.

Pipeline and Technological Stack

In contrast to the neural networks utilized in Agents 2 and 4, Agent 3 relies entirely on rigid mathematical formalisms. It utilizes the Python NumPy and SciPy libraries to perform highly parallelized, pixel-wise algebraic calculations across the massive despeckled covariance matrices. The agent's logic is fundamentally built upon the established principles of planetary

radar polarimetry and the Stokes vector space.

Stokes Parameters and the Circular Polarization Ratio

To fully characterize the partially polarized electromagnetic backscatter observed by the Chandrayaan-2 DFSAR instrument, Agent 3 must first derive the four Stokes parameters (S_1 , S_2 , S_3 , S_4). Assuming a circular transmission basis (which is standard for hybrid-polarimetric architectures operating on the Moon), these parameters define the total power and the polarization state of the wave. From the complex covariance matrix, the Stokes parameters are calculated as:

$S_1 = \langle |E_H|^2 \rangle + \langle |E_V|^2 \rangle$, $S_2 = \langle |E_H|^2 \rangle - \langle |E_V|^2 \rangle$, $S_3 = 2 \text{Re}\langle E_H E_V^* \rangle$, $S_4 = -2 \text{Im}\langle E_H E_V^* \rangle$
Here, S_1 represents the total backscattered power, while S_2 , S_3 , and S_4 describe the polarization ellipse. From these fundamental vectors, the agent calculates the Circular Polarization Ratio (CPR), which is the primary diagnostic tool historically used for isolating ice signatures in planetary science. In a circular basis, CPR is defined as the ratio of the same-sense circular polarization to the opposite-sense circular polarization:

$$\text{CPR} = (S_1 - S_4) / (S_1 + S_4)$$

High CPR (values generally greater than 1) is strongly associated with the volume scattering inherent to water-ice trapped within a cryogenic regolith matrix. However, radar astronomy is plagued by a fundamental ambiguity: high CPR is also generated by wavelength-scale surface roughness, such as blocky impact ejecta and jagged rocks that induce dihedral (double-bounce) reflections.

The m-chi Decomposition

To resolve the ambiguity between rock and ice, Agent 3 executes the m- χ (m-chi) decomposition, a compact polarimetric technique pioneered by R. Keith Raney. The m- χ decomposition maps the Stokes parameters onto the Poincaré sphere, characterizing the field using two key variables: the Degree of Polarization (m) and the Poincaré ellipticity parameter (χ).

The Degree of Polarization, which measures the portion of the electromagnetic wave that remains polarized after interacting with the lunar surface, is calculated as :

$$m = \sqrt{(S_2^2 + S_3^2 + S_4^2)} / S_1$$

The ellipticity parameter χ , which determines whether the bounce is odd or even, is derived through the relationship :

$$\sin(2\chi) = -S_4 / (m S_1)$$

Agent 3 utilizes these variables to decompose the total backscatter into three distinct physical scattering mechanisms, mapping them into a Red-Green-Blue (RGB) color space for downstream tensor ingestion :

- **Red (Even/Double-Bounce Scattering):** $R = \sqrt{(m S_1 (1 + \sin 2\chi)) / 2}$
- **Green (Random/Volumetric Scattering):** $G = \sqrt{(S_1 (1 - m))}$
- **Blue (Odd/Surface Scattering):** $B = \sqrt{(m S_1 (1 - \sin 2\chi)) / 2}$

By breaking down the signal mathematically, the agent provides a multi-dimensional physical feature space that allows subsequent AI models to definitively distinguish between a rough, rocky crater wall (dominated by the Red double-bounce component) and an ancient, deeply buried ice slab (dominated by the Green volumetric scattering component).

Internal Operational Flow

The operation of Agent 3 is deterministic, ensuring absolute mathematical consistency across the dataset.

Initiation occurs when the agent receives the file pointers for the clean L-band and S-band C_2 covariance tensors generated by Agent 2.

Execution proceeds through highly optimized, vectorized array operations. The agent steps through the data, calculating the S_1 through S_4 Stokes parameters for both the L-band and S-band frequencies. It subsequently computes the L-band CPR, the S-band CPR, the Degree of Polarization (m), and the three isolated m- χ components (Odd, Even, Volume). Furthermore, the agent integrates the recent empirical findings of Sinha et al. (2026), flagging pixels that meet the strict physical threshold for doubly shadowed crater ice: CPR greater than 1 coupled with a Degree of Polarization strictly lower than 0.13.

The output phase consolidates these calculations. The agent generates a dense, mathematically deterministic "Polarimetric Feature Tensor." This multi-layered array stacks the CPR, m, and m- χ scattering mechanisms, providing the ultimate physical dataset ready for machine learning segmentation.

Learning Resource Category	Specific Reference / Link	Application within Agent 3 Pipeline
Radar Polarimetry	"The m-chi decomposition of hybrid dual-polarimetric radar data" (Raney et al., 2012)	The seminal mathematical derivation of the m- χ parameters from Stokes vectors.
Lunar Ice Diagnostics	Hackathon Proposal Analysis: Physical Thresholds	Implementing the CPR > 1 and DOP < 0.13 logic specific to Faustini Crater ice detection.
Poincaré Sphere	Thorlabs / IEEE Geoscience Letters	Visualizing and calculating polarization states using m and χ .

Chapter 4: Agent 4 - The Weakly-Supervised Semantic Segmentation Agent

Agent 4 represents the core analytical intelligence of the LunarSight framework. Its mandate is to ingest the multi-channel polarimetric feature tensor generated by Agent 3, synthesize it with the topographic data from Agent 1, and output a highly accurate, binary classification map indicating the precise geospatial boundaries of subsurface ice deposits versus barren lunar rock.

Pipeline and Technological Stack

In terrestrial remote sensing, fully supervised semantic segmentation requires tens of thousands of dense, pixel-perfect ground truth masks. However, in lunar exploration, there is an absolute absence of in-situ physical drilling data within the Permanently Shadowed Regions of the South Pole. Because exact pixel-level ground truth for volumetric ice simply does not exist, training a traditional supervised model is impossible. Agent 4 circumvents this insurmountable obstacle by employing a Weakly-Supervised Cross-Modal Learning paradigm.

The agent utilizes the **Segmentation Models PyTorch (SMP)** library, a highly modular framework that decouples the network architecture from the feature extractor (encoder). This modularity allows developers to rapidly swap configurations based on performance. Agent 4 deploys a classic U-Net architecture—renowned for its spatial precision in biomedical and remote sensing applications due to its symmetric encoder-decoder structure and skip connections—backed by an encoder (such as EfficientNet or ResNet) that has been pre-trained on massive terrestrial datasets like ImageNet. This transfer learning approach allows the network to rapidly converge on the generalized textures and topological features of lunar impact craters without requiring a massive dataset trained from scratch.

Weak Supervision and Physical Priors

Weakly-supervised segmentation reduces the labeling burden by replacing dense, hand-drawn masks with coarse, incomplete, or algorithmically generated "pseudo-labels". In this architecture, sparse pseudo-labels are algorithmically generated using the rigid physical priors calculated by Agent 3.

Relying on the empirical diagnostic criteria established by Sinha et al. (2026), the agent's logic hardcodes definitive "ice seeds" only at pixels that satisfy three extreme conditions simultaneously :

1. L-band CPR > 1 (Indicating deep volume scattering or extreme roughness).
2. S-band CPR > 1 (Indicating shallow volume scattering or surface roughness).
3. Degree of Polarization (DOP) < 0.13 (Crucially filtering out the spurious dihedral returns from blocky rocks).

Pixels meeting these rigorous physical criteria are marked as a definitive 1 (Ice). Conversely, sunlit equatorial pixels exhibiting high $m\text{-}\chi$ odd-bounce characteristics (indicating bare regolith) are marked as a definitive 0 (Rock). The vast majority of the crater surface, which remains ambiguous, is left unlabeled as a NaN value.

During the training cycle, the U-Net acts as a highly advanced spatial interpolator. By analyzing the contextual multi-modal tensor (intensity, topographic slope, $m\text{-}\chi$ components) surrounding the sparse seeds, the network learns the latent polarimetric textures of genuine ice. Through mechanisms like Conditional Random Fields (CRFs) or iterative self-training, the network confidently generates dense predictions across the ambiguous boundary regions, effectively expanding the sparse pseudo-labels into a complete, dense semantic mask covering the entire crater floor.

Internal Operational Flow

The internal operation of Agent 4 bridges the gap between physics and computer vision.

The initiation phase begins when the agent loads both the Polarimetric Feature Tensor (from Agent 3) and the LOLA DEM/Slope tensor (from Agent 1).

During execution, the agent first runs a heuristic mask generator. This script applies the CPR > 1 and DOP < 0.13 logic across the tensor to create the sparse training seeds. The SMP U-Net model is then initialized with its ImageNet pre-trained weights. The model is fine-tuned iteratively, learning to correlate the sparse ice/rock labels with the broader radar cross-sections. Once the cross-entropy loss function converges, the network performs a final forward inference pass over the entire Faustini Crater dataset.

In the output phase, the agent produces a "Binary Ice Mask" (a highly detailed spatial grid designating the exact Ice vs. Rock boundaries). Accompanying this mask is a probabilistic

confidence map, serving as the ultimate geological target destination for the autonomous rover.

Learning Resource Category	Specific Reference / Link	Application within Agent 4 Pipeline
Neural Network Architecture	Segmentation Models PyTorch (SMP) Documentation	Initializing the U-Net architecture, selecting encoders, and configuring training loops.
Weakly Supervised Learning	Pseudo-labeling strategies in remote sensing (iSAGE framework)	Understanding the transition from sparse, targeted pixel seeds to dense semantic maps via CRFs.
Prior Generation	Hackathon Presentation Analysis	The scientific justification for using $DOP < 0.13$ as a strict pseudo-label seed for ice.

Chapter 5: Agent 5 - The Terramechanic-Aware Spatiotemporal Pathfinding Agent

The isolation and detection of subsurface ice hold no practical value unless an autonomous robotic platform can safely navigate the lunar topography to extract it. The Faustini Crater presents a highly extreme operational environment; the crater spans 41 kilometers in diameter, characterized by steep crater walls with median slopes exceeding 12.3° , leading down to heavily cratered, cryogenic lower floor units exhibiting mass wasting and temperatures stabilizing near 25K . Agent 5 is explicitly tasked with routing the rover from the illuminated crater rim down into the shadowed ice reserves identified by Agent 4, avoiding total mission failure.

Pipeline and Technological Stack

Standard geometric pathfinding algorithms—such as basic Dijkstra or static A^* operating on a rigid grid—are entirely insufficient for planetary robotics. If an algorithm penalizes a 15° slope but ultimately commands the rover to traverse it because alternative routes are mathematically longer, the vehicle will experience catastrophic failure, as standard wheeled rovers (like ISRO's Pragyan) possess strict slope climbing limits bounded near 10° . To overcome this, Agent 5 employs a highly customized *Spatiotemporal Kinodynamic A Algorithm**, implemented via Python's SciPy.spatial and advanced graph-theory network routing libraries. This planner does not simply look at geometry; it integrates live physical models of soil failure and tracking moving solar illumination.

Terramechanics: Slip, Sinkage, and Soil Failure

The most glaring omission in conceptual navigation frameworks is the lack of terramechanical modeling. The lunar surface is covered in unconsolidated, fine-grained regolith. When a wheeled vehicle attempts to ascend a slope, the soil yields, causing severe longitudinal slip and lateral sinkage. A pure geometric analysis is insufficient; Agent 5 calculates a slip-aware cost function $g(n)$ based on classical terramechanics.

The interaction between the rover wheel and the regolith is governed by the **Mohr-Coulomb**

Failure Criterion, which dictates the maximum shear stress ($\tau_{\max}$) the soil can withstand before catastrophic failure occurs :

$$\tau_{\max} = c + \sigma \tan \phi$$

Where c represents the soil cohesion, σ is the applied normal stress from the rover's mass, and ϕ is the internal friction angle of the regolith (approximately 30° to 35° for lunar soil).

As the wheel turns, the actual shear stress (τ) generated as it slips through the regolith is modeled by the **Janosi-Hanamoto equation** :

$$\tau(j) = (c + \sigma \tan \phi)(1 - e^{-j/K})$$

Where j is the soil shear displacement (driven by wheel slip) and K is the shear-deformation modulus.

By integrating these equations across the wheel contact patch, Agent 5 predicts the gross tractive effort and precise drawbar pull of the rover based on the localized Horn's slope derivative (provided by Agent 1). If the slope angle approaches the friction angle of the soil, the required tractive force to climb the slope will exceed $\tau_{\max}$, resulting in 100% slip and complete immobilization. When this threshold is breached, Agent 5 mathematically registers the terrain as an infinitely costly, impenetrable obstacle, refusing to plot a route regardless of distance.

Spatiotemporal Illumination Constraints

Beyond slip, a solar-powered rover cannot simply drive into a Permanently Shadowed Region (PSR) without tracking its battery capacitance and internal thermal limits. The Moon's synodic period lasts approximately 708 hours, creating slow-moving, high-contrast shadows.

Agent 5 models the search space in four dimensions (x , y , θ , t). The spatiotemporal A^* search tracks the moving sun vector across the LOLA DEM. The heuristic function $h(n)$ dynamically manages a virtual thermal and electrical battery, ensuring that the path length—adjusted for the slower velocity caused by terramechanical slip—guarantees the rover can physically reach the ice deposit and return to the sunlit rim before critical thermal depletion occurs in the 25K environment.

Internal Operational Flow

Agent 5 operates as the final arbitrator of mission safety, transforming semantic maps into actionable driving commands.

Initiation is triggered when Agent 5 receives the Binary Ice Mask (from Agent 4) marking the target destination, alongside the LOLA DEM and Horn's Slope array (from Agent 1) providing the environmental map.

Execution begins by discretizing the map into a searchable graph. The agent expands nodes iteratively using the A^* formula $f(n) = g(n) + h(n)$. The cost function $g(n)$ dynamically evaluates every adjacent pixel, applying the Janosi-Hanamoto and Mohr-Coulomb equations to calculate the expected wheel slip based on the specific mass and suspension type of the deployed rover (e.g., a 6-wheel rocker-bogie vs. a 4-wheel active suspension). Simultaneously, the heuristic $h(n)$ evaluates the remaining distance, heavily weighing it against the spatiotemporal movement of lunar shadows to prevent thermal death.

The output phase generates a final, optimal, kinodynamically safe, and thermally viable rover driving traverse. This output is structured as a matrix of geographic coordinates, expected temporal waypoints, and anticipated slip ratios, ready for ingestion by the rover's local mobility

controller.

Learning Resource Category	Specific Reference / Link	Application within Agent 5 Pipeline
Lunar Terramechanics	"Analytical determination of drawbar pull as a function of slip" (Janosi & Hanamoto)	The foundational equations defining shear displacement and wheel slip on deformable terrain.
Soil Failure Models	Mohr-Coulomb Failure Criterion Literature	Calculating the absolute physical limit of slope traversal based on soil cohesion.
Dynamic Routing	Spatiotemporal path planning literature	Implementing 4D search algorithms that track moving solar vectors and manage thermal limits.

Chapter 6: Unified Multi-Agent Orchestration via LangGraph

While Agents 1 through 5 represent highly specialized, incredibly powerful analytical silos, a planetary software architecture requires a robust orchestration layer. This layer must manage vast data flows, trigger sequential execution, handle fault tolerance, and manage complex state handoffs. To achieve this level of system cohesion, the LunarSight architecture employs **LangGraph**, a framework explicitly designed for building resilient, stateful multi-agent applications utilizing graph-based state machines.

Architectural Paradigm: The StateGraph

Traditional Directed Acyclic Graph (DAG) executors force data through rigid, one-way pipelines. In contrast, LangGraph permits cyclical workflows, which are absolutely critical for error recovery, iterative refinement, and autonomous conflict resolution. The entire LunarSight system is modeled as a StateGraph, consisting of three core interconnected components:

1. **State Schema (TypedDict):** At the center of multi-agent design is context engineering. The system relies on a strictly typed, centralized JSON-like state object that represents the shared memory of the entire application. This state dictionary contains keys for data paths (e.g., `raw_data_path`, `c2_matrix_path`), process status flags (`agent_1_status`), system constraints (`rover_max_slope`, `thermal_battery_limit`), and execution metadata. Crucially, instead of passing massive gigabyte-scale tensors directly through memory—which would cause immediate memory fragmentation—the agents pass lightweight file pointers and metadata via the state object, enabling highly efficient handoffs.
2. **Nodes:** Agents 1 through 5 are encapsulated as individual Python functions (nodes) within the graph. When triggered, a node extracts its necessary context from the State, performs its specialized task (e.g., CV-CNN despeckling or A* pathfinding), and writes the output paths and status codes back to the State.
3. **Edges and Conditional Routing:** Edges define the control flow rules between nodes. LunarSight utilizes dynamic conditional edges driven by a central "Supervisor Agent" (a lightweight routing logic block that coordinates the specialized agents).

The Orchestration Flow and Cyclical Conflict Resolution

The orchestration operates under a deterministic Supervisor-Worker pattern.

Phase 1 (Preparation): The workflow is initialized when a human operator injects mission parameters (e.g., target crater coordinates, rover kinematic limits) into the StateGraph. The Supervisor activates Agent 1. Upon completion of downloading and topographic synthesis, Agent 1 updates the state with the paths to the synthesized DEM and raw radar arrays.

Phase 2 (Radar Analysis): The Supervisor detects the completion of Agent 1 and triggers Agent 2. Agent 2 despeckles the complex data, serializes it, and updates the state. The Supervisor then routes to Agent 3, which executes the m-ichi physics calculations, subsequently returning the feature tensor pointers to the state object.

Phase 3 (Segmentation and Routing): Agent 4 is triggered, reading the physical tensors and generating the binary ice mask. The state is updated, providing the final destination coordinates for Agent 5, which then executes the terramechanical pathfinding algorithm.

Cyclical Conflict Resolution: LangGraph's cyclical design enables systemic resilience, preventing the architecture from collapsing upon encountering physical impossibilities. For example, if Agent 5 calculates that no safe path exists into the target ice deposit due to severe wheel slip constraints (i.e., the required Janosi-Hanamoto shear stress exceeds the Mohr-Coulomb soil failure limit across all routes), it updates the state with a path_failure flag. The Supervisor intercepts this flag and utilizes a conditional edge to route the workflow *backwards* to Agent 4. The Supervisor manipulates the state to instruct Agent 4 to lower its confidence threshold or select a secondary, shallower ice deposit that may be more accessible. Agent 4 generates a new map, and Agent 5 attempts routing again. This closed-loop reasoning ensures continuous autonomous operation without requiring human intervention, embodying true multi-agent intelligence.

Chapter 7: Strategic Blueprint for Enhancement and Scale

A critical, expert-level evaluation of the initial LunarSight proposal (submitted by Team Aufheben for the ISRO Bharatiya Antariksh Hackathon 2026) revealed an incredibly strong conceptual framework that was severely marred by presentation flaws, fabricated mathematics, and a lack of specific physical modeling. To successfully build, deploy, and scale this multi-agent system from a conceptual prototype into a reliable, enterprise-grade architecture, strict adherence to the following actionable blueprint is required.

1. Eradicate Fabricated Mathematics and Enforce Scientific Rigor

The most severe vulnerability identified in the initial hackathon iteration was the inclusion of mathematical hallucinations intended merely to mimic the visual aesthetic of scientific complexity. The proposal included entirely arbitrary polynomials (e.g., $f(x)=N^2(x-xh^2)$) and nonsensical kinetic energy formulas (e.g., $K_E=\frac{n^2}{\ln 5a_1+1}+\frac{13}{R}$) disconnected from any radar physics or orbital mechanics. In a mission-critical engineering environment, the inclusion of generative AI hallucinations is a fatal flaw that casts doubt on the authenticity of the entire methodology.

- **Actionable Advice:** All arbitrary equations must be completely excised from the

codebase and documentation. The architecture must strictly implement the genuine physical equations outlined in Chapters 3 and 5—specifically the Raney m- χ i derivations for polarimetry and the Mohr-Coulomb/Janosi-Hanamoto formulas for terramechanics.

- **Actionable Advice:** System architecture metrics must be corrected. The initial document claimed data throughputs measured in fictitious units like "sbps" and "mprs", and processing power in "mpes". These must be replaced with standardized data engineering metrics: Megabytes per second (MB/s) for I/O throughput, TFLOPS (Floating Point Operations Per Second) for AI compute power, and milliseconds (ms) for user latency.

2. Capitalize on Decentralized Cloud GPU Economics

Scaling the LunarSight architecture—specifically the training regimens for the CV-CNN (Agent 2) and the U-Net segmenter (Agent 4) on massive orbital DFSAR swathes—requires immense VRAM to accommodate high batch sizes and high-resolution complex tensors without encountering Out-Of-Memory (OOM) fragmentation. Traditional hyperscalers (such as AWS or Azure) are often economically prohibitive for continuous training cycles.

- **Actionable Advice:** The project must fully leverage decentralized GPU marketplaces, specifically platforms like Vast.ai or Spheron.

GPU Architecture	VRAM	Provider	Estimated On-Demand Rate (\$/hr)
NVIDIA H100 PCIE	80 GB	Vast.ai / Spheron	~\$2.00 - \$2.01
NVIDIA H100 SXM5	80 GB	AWS (p5 instances)	~\$6.88
NVIDIA A100 PCIE	80 GB	Vast.ai	~\$0.51
NVIDIA RTX 4090	24 GB	Vast.ai	~\$0.35

Data aggregated from 2026 cloud pricing indices.

- **Actionable Advice:** As demonstrated by the cost comparison, an enterprise-grade NVIDIA H100 PCIE can be secured on decentralized networks for approximately \$2.00 per hour, a fraction of the AWS cost. By securing an initial budget of merely \$500 USD, the development team can provision 250 hours of uninterrupted H100 compute time. For smaller rapid prototyping runs, the RTX 4090 provides exceptional performance at a highly expendable \$0.35 per hour. This economic efficiency proves that high-fidelity planetary AI models can be developed entirely independent of multi-million-dollar traditional hardware budgets.

3. Hardcode the Latest Empirical Radar Thresholds

The ultimate success of the weakly-supervised segmentation model (Agent 4) relies entirely on the scientific accuracy of its sparse pseudo-labels. If the AI is fed bad priors, it will simply learn to confidently identify rock as ice.

- **Actionable Advice:** The pseudo-labeling algorithm must be structurally anchored to the most recent empirical literature, specifically the diagnostic criteria established by Sinha et al. (2026) for doubly shadowed craters. The system must enforce a rigid Boolean logic gate: Volumetric ice seeds are generated *if and only if* L-band CPR > 1 AND DOP < 0.13. Hardcoding this threshold ensures the model is immune to the false positive signals generated by blocky anorthosite ejecta, securing the genuine high-confidence mapping demanded by the architecture.

4. Continuous Integration of Spatiotemporal Dynamics

The incorporation of Agent 5 represents a major leap over standard geometric planning. However, to scale the system for an actual live mission (e.g., controlling architectures like ISRO's Pragyan or NASA's VIPER), the planner must act as a dynamic digital twin.

- **Actionable Advice:** The spatiotemporal A* algorithm must integrate a localized, real-time thermal management heuristic. The node expansion function must carry a dynamic electrical battery capacitance variable that depletes at a rate proportional to the severity of the calculated Janosi-Hanamoto wheel slip (as higher slip demands exponentially higher torque and power draw from the motors).
- **Actionable Advice:** By ensuring the LangGraph orchestrator continuously loops updated 708-hour LOLA illumination models into Agent 5, the rover can safely navigate the cryogenic boundaries of Faustini Crater, ensuring total mission success within the rigid physical realities of the lunar environment.

Works cited

1. GLOBAL MINI-RF S-BAND CPR AND M-CHI DECOMPOSITION OBSERVATIONS OF THE MOON, Joshua T. S. Cahill¹, D. Benjamin J. Bussey¹, G. W - Lunar and Planetary Institute, <https://www.lpi.usra.edu/meetings/lpsc2012/pdf/2590.pdf> 2. Chandrayaan-2 - ISRO, https://www.isro.gov.in/Chandrayaan2_science.html 3. ANOMALOUS CRATERS AT THE LUNAR SOUTH POLE IMAGED USING CHANDRAYAAN-2 DFSAR. P. McVann¹, R. C. Ghail¹, G. Gallardo i Peres² and P, <https://www.hou.usra.edu/meetings/lpsc2025/pdf/1580.pdf> 4. Multi-fidelity Gaussian processes for terramechanical modeling - mediaTUM, <https://mediatum.ub.tum.de/doc/1783492/1783492.pdf> 5. Advanced traction system design for pressurized rovers for lunar surface exploration - WebThesis, <https://webthesis.biblio.polito.it/40355/1/tesi.pdf> 6. Chandrayaan-2 PDS4 Data User Guide | PDF | Zip (File Format) - Scribd, <https://www.scribd.com/document/827566443/Chandrayaan-2-IIRS-PDS4-Data-Products-Users-Guide> 7. Chandrayaan-2 - PRADAN - ISSDC, <https://pradan.issdc.gov.in/ch2/> 8. Payloads Data& Science - ISSDC, https://www.issdc.gov.in/docs/ch2/hand_book_-_payloads_data_and_science.pdf 9. LRO LOLA - PDS Geosciences Node Data and Services, <https://pds-geosciences.wustl.edu/missions/lro/lola.htm> 10. Astropedia - Moon LRO LOLA DEM 118m, https://astrogeology.usgs.gov/search/map/moon/lro_lola_dem_118m 11. High-resolution Lunar Topography (SLDEM2015) - PGDA - NASA, <https://pgda.gsfc.nasa.gov/products/54> 12. Terrain Algorithms from Scratch - Signals & Pixels, <https://www.aazuspan.dev/blog/terrain-algorithms-from-scratch/> 13. Report on the generation and analysis of DEMs for spatial modelling - The Macaulay Land Use Research Institute, <https://macaulay.webarchive.hutton.ac.uk/LADSS/documents/DEMs-for-spatial-modelling.pdf> 14. Modeling Slope in a Geographic Information System - ScholarWorks@UARK, <https://scholarworks.uark.edu/context/jaas/article/1578/viewcontent/v58a16.pdf> 15. Slope and aspect methods — xDEM, https://xdem.readthedocs.io/en/stable/advanced_examples/plot_slope_methods.html 16. Terrain Analysis: Using Slope Algorithms, https://www.geo.utexas.edu/courses/371c/project/2018F/Whittington_GIS_project.pdf 17. Multidisciplinary Design Optimization for a Solar-Powered Exploration Rover Considering the

Restricted Power Requirement - MDPI, <https://www.mdpi.com/1996-1073/13/24/6652> 18. Apollo 11 Landing | More than Maps - OS Docs!, <https://docs.os.uk/more-than-maps/geographic-data-visualisation/geodataviz-assets/how-did-i-make-that/apollo-11-landing> 19. GEOLOGY-DERIVED SUPPORTIVE MAPPING PRODUCTS IN LMAP: LANDSCAPE, IMPACT CRATERS, AND SLOPE CHARACTERIZATION. J. A. Skinner, Jr.1., <https://www.hou.usra.edu/meetings/lpsc2026/pdf/1913.pdf> 20. Lunar Polar Illumination - PGDA - NASA, <https://pgda.gsfc.nasa.gov/products/69> 21. Synthetic aperture radar raster types | ArcGIS Pro documentation - Esri, <https://doc.esri.com/en/arcgis-pro/latest/help/data/imagery/sar-raster-types.html> 22. Despeckling of SAR Images Using Residual Twin CNN and Multi-Resolution Attention Mechanism - MDPI, <https://www.mdpi.com/2072-4292/15/14/3698> 23. Complex-Valued 2D-3D Hybrid Convolutional Neural Network with Attention Mechanism for PolSAR Image Classification - MDPI, <https://www.mdpi.com/2072-4292/16/16/2908> 24. Complex-Valued Vs. Real-Valued Convolutional Neural Network for Polsar Data Classification, https://elib.dlr.de/193336/1/Complex-Valued_Vs._Real-Valued_Convolutional_Neural_Network_for_Polsar_Data_Classification.pdf 25. GitHub - XinyuanLiao/ComplexNN: A toolbox for using complex-valued standard network modules in PyTorch, including MLP, CNN, RNN, Attention., <https://github.com/XinyuanLiao/ComplexNN> 26. Implementation of Complex Valued Neural Networks in Pytorch - GitHub, <https://github.com/mehdihosseini-moghadam/Complex-Neural-Networks> 27. wavefrontshaping/complexPyTorch: A high-level toolbox for using complex valued neural networks in PyTorch - GitHub, <https://github.com/wavefrontshaping/complexPyTorch> 28. Library to help implement a complex-valued neural network (cvnn) using tensorflow as back-end - GitHub, <https://github.com/NEGU93/cvnn> 29. Deep Learning Based Speckle Filtering for Polarimetric SAR Images. Application to Sentinel-1 A preprint - arXiv, <https://arxiv.org/html/2408.15678v2> 30. Assessment of L-Band SAR Data at Different Polarization Combinations for Crop and Other Landuse Classification - ResearchGate, https://www.researchgate.net/publication/259562760_ASSESSMENT_OF_L-BAND_SAR_DATA_AT_DIFFERENT_POLARIZATION_COMBINATIONS_FOR_CROP_AND_OTHER_LANDUSE_CLASSIFICATION 31. Analysis of Radarsat-2 Full Polarimetric Data for Forest Mapping - University of Calgary, https://www.ucalgary.ca/engo_webdocs/MJC/11.20347_YasserMaghsoudi.pdf 32. Riemannian Complex Hermit Positive Definite Convolution Network for Polarimetric SAR Image Classification - arXiv, <https://arxiv.org/html/2502.08137v1> 33. PolSAR Image Classification using a Hybrid Complex-Valued Network (HybridCVNet), <https://arxiv.org/html/2605.31137v1> 34. Proposed CV-CNN-SE model for PolSAR image classification. - ResearchGate, https://www.researchgate.net/figure/Proposed-CV-CNN-SE-model-for-PolSAR-image-classification_fig1_374888593 35. The m-chi decomposition of hybrid dual-polarimetric radar data with application to lunar craters | Request PDF - ResearchGate, https://www.researchgate.net/publication/258663012_The_m-chi_decomposition_of_hybrid_dual-polarimetric_radar_data_with_application_to_lunar_craters 36. The m-chi decomposition of hybrid dual-polarimetric radar data - ResearchGate, https://www.researchgate.net/publication/261266294_The_m-chi_decomposition_of_hybrid_dual-polarimetric_radar_data 37. CHARACTERIZATION OF LUNAR CRATER EJECTA DEPOSITS USING RADAR DATA FROM THE MINI-RF INSTRUMENT ON LRO. G.W. Patterson, J.T.S. Cah, <https://www.lpi.usra.edu/meetings/lpsc2013/pdf/2380.pdf> 38. M-chi Decomposition of Imperfect Hybrid Dual-Polarimetric Radar Data - ESA Earth Online, https://earth.esa.int/eogateway/documents/20142/0/2_Raney_Methods.pdf 39. Full article:

Monitoring Crops Using Compact Polarimetry and the RADARSAT Constellation Mission - Taylor & Francis, <https://www.tandfonline.com/doi/full/10.1080/07038992.2022.2121271> 40. Freeman-Durden Decomposition (Quad-Pol) m-chi Decomposition (Compact Hybrid-Pol) - GRSS-IEEE, https://www.grss-ieee.org/wp-content/uploads/2010/03/ngrs_SeptForWeb.pdf 41. Investigating the Retention of Solar Wind Implanted Helium-3 on the Moon from the Analysis of Multi-Wavelength Remote Sensing Data - MDPI, <https://www.mdpi.com/2072-4292/12/20/3350> 42. POLARIMETRIC MODELING OF DUAL FREQUENCY SAR DATA FOR CHARACTERIZATION OF THE LUNAR SURFACE - Essay, https://essay.utwente.nl/fileshare/file/89538/89538_Singh.pdf 43. iSAGE: A Human-in-the-Loop Framework for Remote Sensing Semantic Segmentation via Sparse Point Supervision - arXiv, <https://arxiv.org/html/2606.10136v1> 44. APRIL-MedSeg: A Modular Medical Image Segmentation Toolbox Embracing Modern Paradigms - arXiv, <https://arxiv.org/html/2606.30577v1> 45. Welcome to segmentation_models_pytorch's documentation! - segmentation-models-pytorch, <https://segmentation-models-pytorch.readthedocs.io/en/latest/> 46. segmentation_models_pytorch/docs/insights.rst at main · qubvel-org/segmentation_models_pytorch · GitHub, https://github.com/qubvel/segmentation_models_pytorch/blob/master/docs/insights.rst 47. Full article: AI4LUC: deep learning and automated mask labelling to support land use and land cover mapping in the Cerrado biome - Taylor & Francis, <https://www.tandfonline.com/doi/full/10.1080/2150704X.2024.2382845> 48. segmentation_models_pytorch/docs/quickstart.rst at main · qubvel-org/segmentation_models_pytorch · GitHub, https://github.com/qubvel-org/segmentation_models_pytorch/blob/main/docs/quickstart.rst 49. Weakly Supervised Semantic Segmentation of Remote Sensing Images Using Siamese Affinity Network - MDPI, <https://www.mdpi.com/2072-4292/17/5/808> 50. A Systematic Study on Pretraining Strategies for Low-Label Remote Sensing Image Semantic Segmentation - PMC, <https://pmc.ncbi.nlm.nih.gov/articles/PMC12944012/> 51. themozel/segmentation_models_pytorch: Segmentation models with pretrained backbones. PyTorch. - GitHub, https://github.com/themozel/segmentation_models_pytorch 52. Planetary Mission Preplanning with Heading-Specific Slope and Distance from Shadow, https://www.ri.cmu.edu/app/uploads/2017/06/Final_Thesis_pub.pdf 53. Building Multi-Agent Systems with LangGraph | by Clearwater Analytics Engineering, <https://medium.com/cwan-engineering/building-multi-agent-systems-with-langgraph-04f90f312b8e> 54. (PDF) A Multiscale Convolution SAR Image Target Recognition Method Based on Complex-Valued Neural Networks - ResearchGate, https://www.researchgate.net/publication/390686066_A_Multiscale_Convolution_SAR_Image_Target_Recognition_Method_Based_on_Complex-Valued_Neural_Networks 55. A Dynamic Illumination-Constrained Spatio-Temporal A* Algorithm for Path Planning in Lunar South Pole Exploration - MDPI, <https://www.mdpi.com/2072-4292/18/2/310> 56. A Comprehensive Review of Path-Planning Algorithms for Planetary Rover Exploration, <https://www.mdpi.com/2072-4292/17/11/1924> 57. Development of a New Pressure-Sinkage Model for Rover Wheel-Lunar Soil Interaction based on Dimensional Analysis and Bevameter Tests - Journal of Astronomy and Space Sciences, https://www.janss.kr/archive/view_article?pid=jass-38-4-237 58. Terramechanics - Keck Institute for Space Studies, <https://www.kiss.caltech.edu/workshops/xTerra/presentations1/iagnemma.pdf> 59. Grouser's Effect in Tracked Vehicle Multibody Dynamics with Deformable Terrain Contact Model - MDPI, <https://www.mdpi.com/2076-3417/10/18/6581> 60. RoverDevKit: An open,

physics-grounded tradespace toolkit for conceptual design of lunar micro-rovers - arXiv, <https://arxiv.org/pdf/2606.21755> 61. Scalability of Classical Terramechanics Models for Lightweight Vehicle Applications Incorporating Stochastic Modeling and Uncertainty Propagation - DTIC, <https://apps.dtic.mil/sti/pdfs/ADA592072.pdf> 62. AN ANALYSIS OF THE DRAWBAR PULL VS SLIP RELATIONSHIP FOR TRACK LAYING VEHICLES - DTIC, <https://apps.dtic.mil/sti/citations/tr/AD0275858> 63. How to Coordinate Multiple AI Agents: The Definitive Guide for 2026 - Developers Digest, <https://www.developersdigest.tech/blog/how-to-coordinate-multiple-ai-agents> 64. Conceptual Guides - LangGraph, <https://www.baihezi.com/mirrors/langgraph/concepts/index.html> 65. How to build a multi-agent system using Elasticsearch and LangGraph, <https://www.elastic.co/search-labs/blog/multi-agent-system-llm-agents-elasticsearch-langgraph> 66. Build multi-agent systems with LangGraph and Amazon Bedrock | Artificial Intelligence, <https://aws.amazon.com/blogs/machine-learning/build-multi-agent-systems-with-langgraph-and-amazon-bedrock/> 67. Build a LangGraph Multi-Agent system in 20 Minutes with AgentControl - LaunchDarkly, <https://launchdarkly.com/docs/tutorials/agents-langgraph> 68. How to Build the Ultimate AI Automation with Multi-Agent Collaboration - LangChain, <https://www.langchain.com/blog/how-to-build-the-ultimate-ai-automation-with-multi-agent-collaboration> 69. LangGraph: Multi-Agent Workflows - LangChain, <https://www.langchain.com/blog/langgraph-multi-agent-workflows> 70. Multi-agent - Docs by LangChain, <https://docs.langchain.com/oss/python/langchain/multi-agent> 71. LangGraph — Multi-Agent Systems (MAS) | by Shuvrajyoti Debroy | Medium, <https://medium.com/@shuv.sdr/langgraph-multi-agent-systems-mas-a30166b07691> 72. FareedKhan-dev/contextual-engineering-guide: Implementation of contextual engineering pipeline with LangChain and LangGraph Agents - GitHub, <https://github.com/FareedKhan-dev/contextual-engineering-guide> 73. Build and Evaluate Multi-Agent Systems with Snowflake and LangGraph, <https://www.snowflake.com/en/developers/guides/build-and-evaluate-agents-with-langgraph-and-snowflake/> 74. langgraph_supervisor - LangChain Reference, <https://reference.langchain.com/python/langgraph-supervisor> 75. Building a Multi-Agent System with LangGraph: Supervisor and, <https://callsphere.ai/blog/langgraph-multi-agent-system-supervisor-worker-patterns> 76. How are you handling context sharing in Multi-Agent-Systems(MAS)? Looking for alternatives to rigid JSON states : r/LangGraph - Reddit, https://www.reddit.com/r/LangGraph/comments/1qug9f5/how_are_you_handling_context_sharing_in/